Corrections des questions

TP compute shaders (fondamentaux, simulations spatiales, many lights)

Document enseignant, à ne pas distribuer avec les énoncés

Ce document répond aux questions des trois énoncés. Les consignes du type « écrire le shader » n'y figurent pas : seules les justifications demandées explicitement sont corrigées.

Table des matières

1 TP 1, exercice 1 : addition vectorielle **2**

2 TP 1, exercice 2 : Game of Life **2**
- 2.1 Version A . . . 2
- 2.2 Version B . . . 3
- 2.3 Version C, tuile et halo . . . 3
- 2.4 Réduction du nombre de cellules, cas $D_t = 0$ . . . 4

3 TP 1, exercice 3 : réduction **4**
- 3.1 Version entrelacée . . . 4
- 3.2 Version par moitiés . . . 4
- 3.3 Multi-passes . . . 5
- 3.4 Égalité bit à bit avec la somme CPU . . . 5

4 TP 2, exercice 1 : boids, taille de cellule **5**

5 TP 2, exercice 2 : particules **6**

6 TP 3, exercice 1 : culling **6**

7 TP 3, exercice 2 : tranches linéaires ou logarithmiques **7**

8 TP 3, exercice 3 : affectation lumière vers cluster **7**

9 TP 3, exercice 4 : éclairage **8**

1 TP 1, exercice 1 : addition vectorielle

1. **Identifiant d'une invocation.**

   Réponse attendue

   C'est gl_GlobalInvocationID.x. Son développement est

   $$\texttt{gl_WorkGroupID.x} \times \texttt{gl_WorkGroupSize.x} + \texttt{gl_LocalInvocationID.x}.$$

   Avec local_size_x = 1024, cela vaut `gl_WorkGroupID.x` × 1024 + `gl_LocalInvocationID.x`.

2. $n = 100003$**.**

   Réponse attendue

   Nombre de groupes : $\lceil 100003/1024 \rceil = \lceil 97{,}658\ldots \rceil = 98$.
   Invocations lancées : $98 \times 1024 = 100352$.
   Invocations inutiles : $100352 - 100003 = 349$.

3. **Pourquoi une invocation inutile ne touche pas aux SSBO.**

   Réponse attendue

   Le dispatch arrondit au multiple de la taille locale : ces invocations n'ont pas d'élément. Lire $A[i]$ ou $B[i]$ avec $i \geq n$ sort du tableau transféré (lecture hors bornes). Écrire $C[i]$ déborde du buffer de sortie, ou écrase la mémoire d'un autre objet si le buffer a été alloué plus grand. La garde if (i >= n) return; est donc obligatoire avant tout accès.

4. **Barrière entre invocations.**

   Réponse attendue

   Non. Chaque invocation lit deux cases qui lui sont propres et écrit une case que personne d'autre ne lit dans ce dispatch. Il n'y a ni mémoire shared, ni dépendance producteur/-consommateur entre invocations. La seule barrière utile est celle de l'hôte, après le dispatch, avant glGetBufferSubData : GL_BUFFER_UPDATE_BARRIER_BIT.

5. **`readonly` et `writeonly`.**

   Réponse attendue

   On s'engage à ne jamais écrire dans A et B, et à ne jamais lire C dans ce shader. Le compilateur peut s'en servir (pas de relecture pour vérifier une écriture, pas de conservation d'une valeur écrite). Violer le qualificateur est un comportement indéfini : le résultat n'est plus fiable, même si le code « a l'air » de fonctionner sur une carte donnée.

2 TP 1, exercice 2 : Game of Life

2.1 Version A

1. **Indice linéaire.**

Réponse attendue

Pour une cellule (x, y) et une largeur width :

$$i = x + y \times \texttt{width}.$$

Les coordonnées viennent de gl_GlobalInvocationID.xy, avec une garde $x \geq \texttt{width}$ ou $y \geq \texttt{height}$.

2. **Échange des buffers plutôt qu'une copie.**

Réponse attendue

Après le dispatch, next contient la génération $t{+}1$ et current la génération t. Échanger les deux bindings (ou les deux identifiants de buffer) suffit pour que la génération suivante lise $t{+}1$. Recopier next vers current ferait le même travail en $O(\texttt{width}\times\texttt{height})$ lectures et écritures, plus une barrière, sans changer le résultat. Le ping-pong ne déplace aucune cellule.

2.2 Version B

Réponse attendue

Sur une grille 3×3, le modulo ramène chacun des huit décalages $(\Delta x, \Delta y) \in \{-1, 0, 1\}^2 \setminus \{(0, 0)\}$ sur une case distincte de la grille. Une cellule a donc bien huit voisines, qui sont toutes les autres cases. Une voisine hors grille n'est plus traitée comme morte : son indice est $((x + \Delta x) \bmod w,\ (y + \Delta y) \bmod h)$, avec un modulo positif.

2.3 Version C, tuile et halo

1. **Indices globaux de `tile`.**

Réponse attendue

Le groupe (g_x, g_y) couvre les cellules de sortie $[16g_x,\ 16g_x + 16) \times [16g_y,\ 16g_y + 16)$. Le halo décale l'origine d'une case :

$$\texttt{tile}[lx][ly] \longleftrightarrow (16g_x - 1 + lx,\ 16g_y - 1 + ly),$$

avec $0 \leq lx, ly < 18$. La tuile utile est tile $[1..16][1..16]$.

2. **Où placer `barrier()`.**

Réponse attendue

Une barrière après le chargement coopératif, avant toute lecture de tile pour le comptage. Elle publie les écritures shared des 256 invocations. Sans elle, une invocation peut lire une case de halo encore non écrite par une voisine du groupe. Il n'y a pas de seconde barrière : après ce point, plus personne n'écrit dans tile .

3. **Invocations hors grille.**

Réponse attendue

Deux raisons, qui se cumulent.

— barrier () est collective : si une invocation fait return avant, le reste du groupe l'attend indéfiniment (comportement indéfini). La garde de sortie se place après la barrière.

— Ces invocations servent au chargement. 324 cases pour 256 invocations : une partie du halo n'est chargée que par des invocations qui, elles, ne produisent pas de cellule. Les sortir du chargement laisserait des cases de tile non initialisées.

2.4 Réduction du nombre de cellules, cas $D_t = 0$

Réponse attendue

$D_t = 0$ signifie que l'état $t{+}1$ est identique à l'état t : le motif est un point fixe (nature morte). Ce n'est pas un test de périodicité. Un oscillateur de période 3 vérifie $D_t \neq 0$, $D_{t+1} \neq 0$ et $D_{t+2} \neq 0$, puis retrouve l'état de départ : aucune de ces étapes n'est un point fixe, et $D_t = 0$ ne se produit jamais.

3 TP 1, exercice 3 : réduction

3.1 Version entrelacée

1. **Pourquoi les barrières sont indispensables ici.**

Réponse attendue

Chaque étape lit des cases de partial écrites par d'autres invocations à l'étape précédente. barrier () assure à la fois l'ordre (les écritures sont finies) et la visibilité dans le groupe. À l'exercice 1, aucune invocation ne lisait le résultat d'une autre : la barrière n'avait rien à publier.
Elles se placent hors du if d'activation. Une invocation qui n'additionne rien doit quand même rencontrer chaque barrier (), sinon le groupe se désynchronise.

2. **Motif des invocations actives.**

Réponse attendue

Au pas de stride $s \in \{1, 2, 4, \ldots\}$, l'invocation t est active si t est multiple de $2s$ (elle additionne partial [t + s] dans partial [t]). Les actives sont de plus en plus espacées : une sur deux, puis une sur quatre, etc. Dans un warp de 32, la plupart des voies deviennent inactives très tôt.

3.2 Version par moitiés

1. **Comparaison des motifs.**

Réponse attendue

Ici, au pas $s = n/2, n/4, \ldots, 1$, les invocations actives sont exactement $t < s$: un intervalle contigu au début du groupe. Elles restent regroupées jusqu'aux derniers pas. C'est cette version qui garde des invocations actives contiguës le plus longtemps. La version entrelacée les disperse dès le premier pas.

2. **Mesures pour $n = 1024$.**

Réponse attendue

Il n'y a pas de classement imposé par l'énoncé : les deux versions font le même nombre d'additions ($\sum_k n/2^{k+1} = n-1$). Sur un GPU, la version par moitiés est en général plus régulière (moins de divergence de warp, accès shared plus groupés). L'écart sur un seul groupe de 1024 est souvent faible et peut changer de signe selon le matériel, le driver et le fait que le dispatch est minuscule devant le coût de lancement. On attend un tableau (moyenne sur 500 itérations, après warm-up) et un commentaire sur la stabilité, pas une valeur universelle.

3.3 Multi-passes

Réponse attendue

barrier() ne synchronise que les invocations d'un même groupe. Il n'existe pas de barrière entre groupes à l'intérieur d'un shader : un groupe ne peut pas attendre que les autres aient écrit leur somme partielle. Chaque niveau de réduction est donc un glDispatchCompute distinct. Entre deux dispatchs, glMemoryBarrier avec GL_SHADER_STORAGE_BARRIER_BIT rend les écritures SSBO visibles du dispatch suivant.

3.4 Égalité bit à bit avec la somme CPU

Réponse attendue

L'addition flottante n'est pas associative. La réduction GPU parenthèse les termes par paires ; la boucle CPU les accumule de gauche à droite. Pour des valeurs de même signe et d'ordres de grandeur proches, l'écart reste de l'ordre de quelques ulp et une tolérance relative suffit. Dès que des termes positifs et négatifs se compensent, l'ordre change les annulations : les bits de la mantisse peuvent différer alors que les deux résultats sont « justes » pour leur ordre d'addition. Comparer les bits n'est donc un critère valable que pour des entrées où l'ordre ne change rien (par exemple des puissances de deux exactes, ou un seul élément).

4 TP 2, exercice 1 : boids, taille de cellule

1. **h minimal.**

Réponse attendue

$$h = R = \max(r_s, r_a, r_c).$$

Un voisin à distance $< R$ a un écart de cellule d'au plus 1 sur chaque axe : s'il était à deux cellules, la distance serait $\geq 2h = 2R > R$. Les 27 cellules ($3 \times 3 \times 3$) couvrent donc les trois règles d'un coup. Une valeur plus petite n'est pas fausse, elle est seulement plus chère (question suivante).

2. **Si $h < \max(r_s, r_a, r_c)$.**

Réponse attendue

Le voisinage de ± 1 cellule ne contient plus tous les voisins à distance $< R$. Il faut explorer, de chaque côté et sur chaque axe, $\lceil R/h \rceil$ cellules. Le nombre de cellules candidates par axe est

$$2\left\lceil \frac{R}{h} \right\rceil + 1,$$

et le voisinage 3D en est le cube. Pour $h = R$, on retrouve 3. Oublier ces cellules produit des forces manquantes : la version grille diverge de la version exhaustive.

5 TP 2, exercice 2 : particules

1. **Complexité en $O(N^2)$.**

Réponse attendue

Chaque pas évalue un couple (i, j) pour tout $j \neq i$: $N(N-1)$ interactions, quel que soit le découpage en groupes. Lancer N invocations (une par particule) ou moins d'invocations qui bouclent davantage ne change pas le nombre de paires. Le dispatch fixe qui fait le travail, pas sa quantité. La grille ne change la complexité que parce qu'elle supprime des paires, celles hors des cellules candidates.

2. **Homogénéité, masses, inversion des types.**

Réponse attendue

Le quotient a la dimension d'un inverse de longueur au carré (longueur au numérateur, longueur au cube au dénominateur). K a donc la dimension d'une longueur^3 par temps^2, comme Gm en gravitation avec des masses unité. L'accélération écrite est proportionnelle à K ; la masse de i n'apparaît pas (elle se simplifie entre $F = m_i a$ et une force proportionnelle à m_i).
Doubler toutes les masses et K : si K joue le rôle de Gm et que la source contribue une seconde fois par sa masse, l'accélération est multipliée par 4. Dans la formule du TP, les masses valent 1 et n'y figurent pas : seul le facteur appliqué à K est observable, et doubler K double $\mathbf{a}_i$.
Inverser tous les q_i ne change aucun produit : $(-q_i)(-q_j) = q_i q_j$. Les accélérations, donc les trajectoires, sont identiques. C'est le critère du test sur 100 pas.

3. **Même loi tronquée, mêmes parois.**

Réponse attendue

La comparaison des positions ne valide la grille que si toute différence vient de la sélection des voisins. Une loi, un ϵ^2, un rayon ou une réflexion aux parois différents fabriquent un écart même avec un voisinage parfait. La réflexion fait partie de l'état : une particule qui rebondit mal change de cellule au pas suivant, et l'erreur s'accumule.

6 TP 3, exercice 1 : culling

1. **Ne pas écrire au-delà de `numLights`.**

Réponse attendue

atomicAdd(visibleCount, 1u) renvoie l'ancien compteur, qui sert d'indice. Comme chaque lumière s'insère au plus une fois, l'indice est dans $[0, \texttt{numLights})$. Le tableau visible est dimensionné à numLights, donc l'écriture est dans les bornes. La garde if (slot < numLights) reste le réflexe correct si cette hypothèse change.

2. **Réinitialiser `visibleCount` et `visible[]`.**

Réponse attendue

atomicAdd part de la valeur déjà en mémoire. Sans remise à zéro, le compteur grandit d'une image à l'autre et finit par dépasser numLights. Les anciens indices resteraient aussi en place au-delà du nouveau compteur ; toute lecture qui oublie la borne visibleCount retraiterait des lumières de l'image précédente. Le clear est un dispatch (ou clearSSBO) distinct, suivi de GL_SHADER_STORAGE_BARRIER_BIT avant le culling : sinon des invocations lisent un compteur encore non remis à zéro.

3. **Pourquoi un dispatch séparé du clustering.**

Réponse attendue

Le clustering parcourt visible [0 .. visibleCount). Cette plage n'est connue qu'une fois que toutes les invocations de culling ont fini. À l'intérieur d'un même dispatch, une invocation ne voit pas les atomicAdd des autres groupes, et il n'y a pas de barrière globale pour les attendre. Deux dispatchs sont le seul moyen d'obtenir la liste complète avant de s'en servir.

4. **Barrière entre culling et clustering.**

Réponse attendue

glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT) : le lecteur suivant est un compute shader qui relit les SSBO visibleCount et visible [] .

7 TP 3, exercice 2 : tranches linéaires ou logarithmiques

Réponse attendue

Des tranches linéaires suffisent parce que l'objectif est un index de cluster correct et le même des deux côtés (construction des AABB et éclairage), pas une densité optimale. L'union des AABB couvre le frustum dès que les coins des tuiles sont déprojetés aux bornes $z_{\text{near}} + k\,\Delta z$. Un découpage logarithmique resserre les tranches près de la caméra, là où la perspective entasse beaucoup de pixels dans peu de profondeur, et élargit les tranches lointaines. On y gagne en général moins de lumières par cluster proche, au prix d'une fonction d'index un peu plus délicate (et toujours partagée à l'identique par les deux passes). Ce n'est pas demandé pour la correction de l'image.

8 TP 3, exercice 3 : affectation lumière vers cluster

1. **Test conservatif et image finale.**

Réponse attendue

Une lumière en trop est réévaluée dans la passe d'éclairage avec le test de rayon exact $d < r$ et l'atténuation $A(d)$, qui vaut 0 au-delà de r. Elle n'ajoute rien à L_o. Une lumière oubliée, en revanche, ne peut plus être rattrapée. Le test sphère/AABB a donc le droit d'être trop large ; il n'a pas le droit d'être trop strict.

2. **Plusieurs clusters, contrairement à une particule.**

Réponse attendue

Un boid est un point : il tombe dans une seule cellule de la grille. Une lumière est une sphère, et un cluster est un volume. La sphère intersecte autant d'AABB qu'elle en chevauche, souvent plusieurs tuiles et plusieurs tranches. La ranger dans un seul cluster éteindrait son éclairement partout ailleurs.

3. **Insertions concurrentes, ordre non défini.**

Réponse attendue

atomicAdd sur lightCount[c] attribue à chaque invocation un slot distinct. Deux insertions dans la même liste ne s'écrasent pas. L'ordre des indices est celui dans lequel les atomiques ont été servis, sans rapport avec l'indice de lumière ni avec l'ordre du dispatch. La passe d'éclairage somme des contributions : à l'arrondi flottant près, la somme ne dépend pas de l'ordre. L'image de référence se compare donc avec une tolérance, et les ensembles d'indices se comparent sans tenir compte de l'ordre.

4. **Dépassement visible, pas de troncature silencieuse.**

Réponse attendue

Au-delà de maxPerCluster, des lumières qui intersectent le cluster sont jetées. L'image est alors fausse, et rien ne la distingue d'un éclairage correct si le drapeau reste interne. Afficher les clusters en dépassement (en rouge) sépare une erreur de capacité d'une erreur de test. La boucle de lecture reste bornée par min(lightCount[c], maxPerCluster) : le compteur, lui, a continué à croître, et s'en servir tel quel lirait hors de la zone du cluster.

9 TP 3, exercice 4 : éclairage

1. **Barrière `GL_TEXTURE_FETCH_BARRIER_BIT`.**

Réponse attendue

Le dispatch écrit outputImage. Le lecteur suivant est l'affichage, qui échantillonne cette texture (fetch), pas un autre compute via SSBO. Le bit décrit cet accès suivant. Un GL_SHADER_IMAGE_ACCESS_BARRIER_BIT serait le bon choix si la suite relisait l'image par imageLoad.

2. **Taille locale** 16×16**.**

Réponse attendue

Le domaine est l'image : les coordonnées naturelles d'une invocation sont $(x, y) =$ `gl_GlobalInvocationID.xy`, et le dispatch est $\lceil w/16 \rceil \times \lceil h/16 \rceil$. Un groupe 16×16 couvre un bloc compact de pixels, cohérent avec le cache du G-buffer, et compte 256 invocations, un remplissage de groupe usuel.
À l'exercice 1 les données étaient un tableau 1D : une taille locale selon x suffisait, et une seconde dimension n'aurait rien décrit.

3. **Quand la construction coûte plus que ce qu'elle fait gagner.**

Réponse attendue

La construction visite chaque couple (cluster, lumière visible). Elle est rentable seulement si la passe d'éclairage, ensuite, parcourt bien moins de lumières que la référence exhaustive.
Elle ne l'est plus quand presque toutes les lumières tombent dans presque tous les clusters : rayons très grands, lumières réparties sur tout le frustum, tuiles grosses ou N_z petit. Chaque pixel reteste alors presque N lumières, plus le coût de construction, plus la mémoire des listes. Le cas inverse (beaucoup de petites lumières localisées, tuiles et tranches fines) est celui où le nombre moyen de lumières par cluster décroche de N et où le gain apparaît.